

Notes 22.04.2025

Loading and Processing Geospatial Data

Step 1: Load Austrian Lines and Border from Shapefiles

We started by loading the Austrian lines and border data from shapefiles using the `geopandas` library.

```
lines_aut = gpd.read_file("qgis/lines_aut.shp")
grenze = gpd.read_file("qgis/Grenze.shp")
```

Step 2: Set Coordinate Reference System (CRS)

We checked if the CRS was missing for the loaded shapefiles and set it to EPSG:31287 (Austria Lambert) if necessary.

```
if lines_aut.crs is None:
    lines_aut.set_crs(epsg=31287, inplace=True)
if grenze.crs is None:
    grenze.set_crs(epsg=31287, inplace=True)
```

Step 3: Reproject Data

We reprojected the data to ensure all datasets have the same CRS.

```
lines_aut = lines_aut.to_crs(epsg=31287)
grenze = grenze.to_crs(epsg=31287)
```

Loading and Processing Original Lines from CSV

Step 4: Load CSV File

We loaded the original lines data from a CSV file.

```
file_path = 'cross_border_lines_qgis_spatial.csv'
df = pd.read_csv(file_path)
```

Step 5: Convert WKT Column to Shapely Geometries

We converted the WKT column in the CSV file to shapely geometries.

```
df['geometry'] = df['wkt'].apply(wkt.loads)
```

Step 6: Create a GeoDataFrame

We created a GeoDataFrame from the CSV data and set the CRS to EPSG:31287.

```
gdf = gpd.GeoDataFrame(df, geometry='geometry')  
gdf.set_crs(epsg=31287, inplace=True)
```

Plotting the Data

Step 7: Plot the Data

We plotted the Austrian lines, border, and original lines using `matplotlib`.

```
fig, ax = plt.subplots(figsize=(12, 12))  
  
grenze.plot(ax=ax, color="none", edgecolor="black", linewidth=1,  
label="Grenze.shp (Border)")  
lines_aut.plot(ax=ax, color="purple", linewidth=2, label="lines_aut.shp  
(Austrian lines)")  
gdf.plot(ax=ax, color="blue", linewidth=1, alpha=0.5, linestyle='--',  
label="Original CSV Lines")  
  
plt.title("Comparison: Austrian Grid (lines_aut) vs Original 380kV Lines")  
plt.xlabel("Easting")  
plt.ylabel("Northing")  
plt.grid(True)  
plt.legend()  
plt.tight_layout()
```

Step 8: Save the Plot as a High-Resolution Image

We saved the plot as a high-resolution image.

```
image_path = "highres_plot.png"  
plt.savefig(image_path, dpi=300)  
plt.close()
```

Creating a PDF with the Plot

Step 9: Create a PDF and Embed the High-Resolution Image

We created a PDF and embedded the high-resolution image into it using the `reportlab` library.

```
pdf_path = "comparison_plot.pdf"
c = canvas.Canvas(pdf_path, pagesize=A4)
c.drawImage(image_path, 50, 400, width=500, height=500)
c.showPage()
c.save()
```

Summary

- Loaded and processed geospatial data from shapefiles and CSV.
- Set and reprojected CRS to ensure consistency.
- Plotted the data with appropriate styling to visualize comparisons.
- Saved the plot as a high-resolution image and embedded it into a PDF.

This workflow allows us to effectively compare different geospatial datasets and present the results in a high-quality format.